

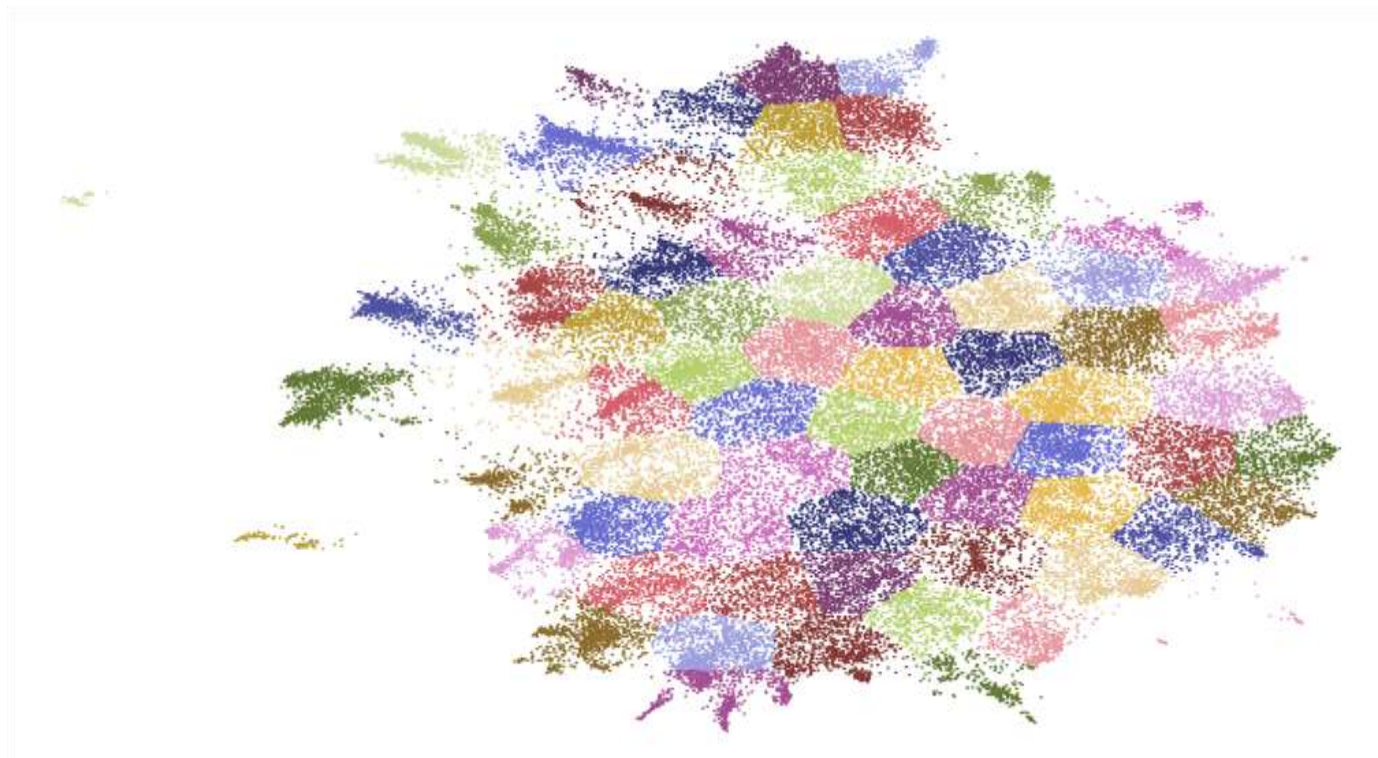
重庆大学计算机专业核心课程《机器学习》

第9章 聚类

授课教师：郑林江

基本概念

□ **聚类**：根据客体属性对一系列未分类的客体进行类别的识别，把一组个体按照相似性归成若干个类，属于**无监督学习方法**。

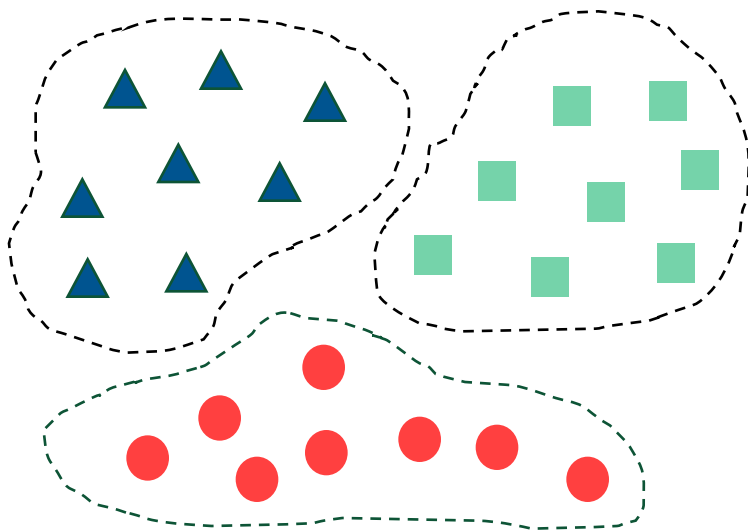


大纲

- 聚类任务
- 性能度量
- 距离计算
- 原型聚类
- 密度聚类
- 层次聚类

聚类任务

- 聚类（clustering）在“无监督学习”任务中研究最多、应用最广
- 聚类目标：将数据集中的样本划分为若干个通常不相交的子集（“簇”，cluster）
- 聚类既可以作为一个单独过程（用于找寻数据内在的分布结构），也可作为分类等其他学习任务的前驱过程



大纲

- 聚类任务
- 性能度量
- 距离计算
- 原型聚类
- 密度聚类
- 层次聚类

性能度量

- 聚类性能度量，亦称为聚类“有效性指标” (validity index)

- 直观来讲：

我们希望“物以类聚”，即同一簇的样本尽可能彼此相似，不同簇的样本尽可能不同。换言之，聚类结果的“簇内相似度” (intra-cluster similarity) 高，且“簇间相似度” (inter-cluster similarity) 低，这样的聚类效果较好

- 聚类性能度量：

- 外部指标 (external index)

将聚类结果与某个“参考模型” (reference model) 进行比较

如 Jaccard 系数，FM 指数，Rand 指数

- 内部指标 (internal index)

直接考察聚类结果而不利用任何参考模型

如 DB指数，Dunn 指数

性能度量

对数据集 $D = \{x_1, x_2, \dots, x_m\}$ ，假定通过聚类得到的簇划分为 $C = \{C_1, C_2, \dots, C_k\}$ ，参考模型给出的簇划分为 $C^* = \{C_1^*, C_2^*, \dots, C_s^*\}$ 相应地，令 λ 与 λ^* 分别表示与 C 和 C^* 对应的簇标记向量。

我们将样本两两配对考虑，定义

$$a = |SS|, SS = \{(x_i, x_j) | \lambda_i = \lambda_j, \lambda_i^* = \lambda_j^*, i < j\}$$

$$b = |SD|, SD = \{(x_i, x_j) | \lambda_i = \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\}$$

$$c = |DS|, DS = \{(x_i, x_j) | \lambda_i \neq \lambda_j, \lambda_i^* = \lambda_j^*, i < j\}$$

$$d = |DD|, DD = \{(x_i, x_j) | \lambda_i \neq \lambda_j, \lambda_i^* \neq \lambda_j^*, i < j\}$$

- ✓ 集合 SS 包含了在 C 中隶属于相同簇且在 C^* 中也隶属于相同簇的样本对。
- ✓ 集合 SD 包含了在 C 中隶属于相同簇但在 C^* 中隶属于不同簇的样本对。

由于每个样本对 (x_i, x_j) ($i < j$) 仅能出现在一个集合中，因此有 $a+b+c+d = m(m-1)/2$ 成立。

性能度量 - 外部指标

□ Jaccard系数 (Jaccard Coefficient, JC)

$$JC = \frac{a}{a+b+c}$$

□ FM指数 (Fowlkes and Mallows Index, FMI)

$$FMI = \sqrt{\frac{a}{a+b} \cdot \frac{a}{a+c}}$$

□ Rand指数 (Rand Index, RI)

$$RI = \frac{2(a+d)}{m(m-1)}$$

[0,1]区间内,
越大越好.

性能度量 - 内部指标

- 考虑聚类结果的簇划分 $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$, 定义簇 C 内样本间的平均距离

$$avg(C) = \frac{2}{|C|(|C|-1)} \sum_{1 \leq i < j \leq |C|} dist(x_i, x_j)$$

簇 C 内样本间的最远距离

$$diam(C) = \max_{1 \leq i < j \leq |C|} dist(x_i, x_j)$$

簇 C_i 与簇 C_j 最近样本间的距离

$$d_{min}(C) = \min_{x_i \in C_i, x_j \in C_j} dist(x_i, x_j)$$

簇 C_i 与簇 C_j 中心点间的距离

$$d_{cen}(C) = dist(\mu_i, \mu_j)$$

性能度量 - 内部指标

□ DB指数 (Davies-Bouldin Index, DBI)

$$DBI = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{avg(C_i) + avg(C_j)}{d_{cen}(\mu_i, \mu_j)} \right)$$

越小越好.

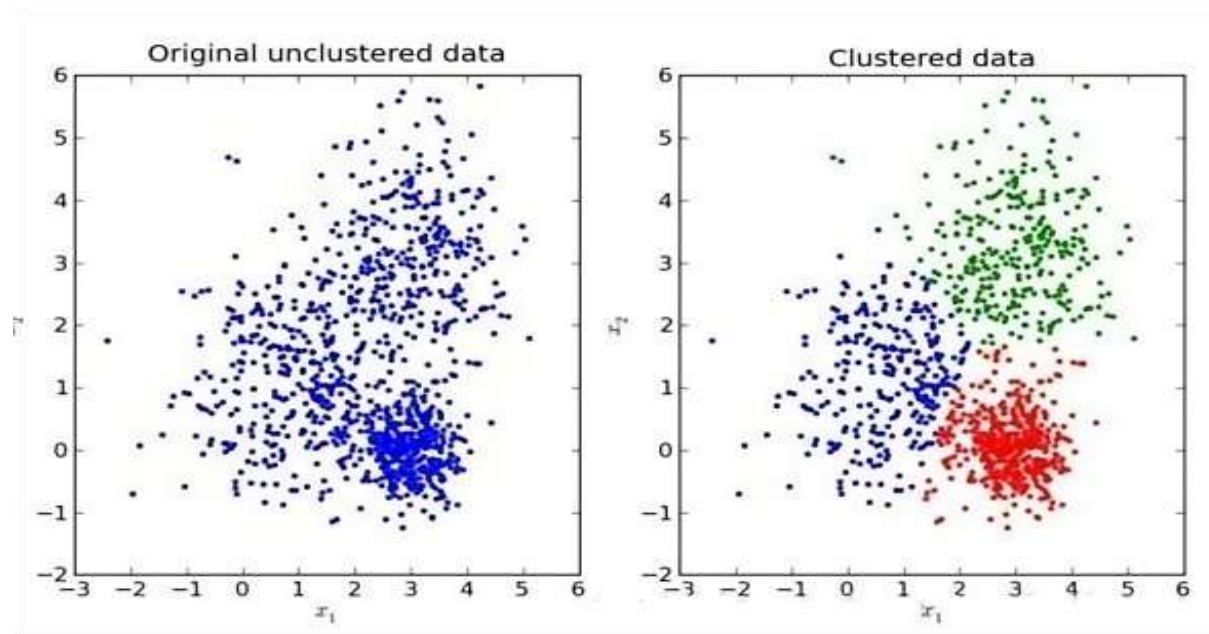
□ Dunn指数 (Dunn Index, DI)

$$DI = \min_{1 \leq i \leq k} \left\{ \min_{j \neq i} \left(\frac{d_{min}(C_i, C_j)}{\max_{1 \leq l \leq k} diam(C_l)} \right) \right\}$$

越大越好.

性能度量

□ 对于无类标的情况，没有唯一的评价指标。对于数据 **凸分布** 的情况只能通过 **类内聚合度**、**类间低耦合** 的原则来作为指导思想，如下图：



性能度量 - 内部指标

- Silhouette Coefficient 轮廓系数(SC):

轮廓系数 (Silhouette coefficient) 适用于实际类别信息未知的情况。对于单个样本, 设 a 是与它同类别中其他样本的平均距离, b 是与它距离最近不同类别中样本的平均距离, 轮廓系数为:

$$s = \frac{b-a}{\max(a,b)}$$



轮廓系数取值范围是 $[-1, 1]$, 同类别样本越距离相近且不同类别样本距离越远, 分数越高。

对于一个样本集合, 它的轮廓系数是所有样本轮廓系数的平均值。

```
1 >>> import numpy as np
2 >>> from sklearn.cluster import KMeans
3 >>> kmeans_model = KMeans(n_clusters=3, random_state=1).fit(X)
4 >>> labels = kmeans_model.labels_
5 >>> metrics.silhouette_score(X, labels, metric='euclidean')
6 ...
7 0.55...
```

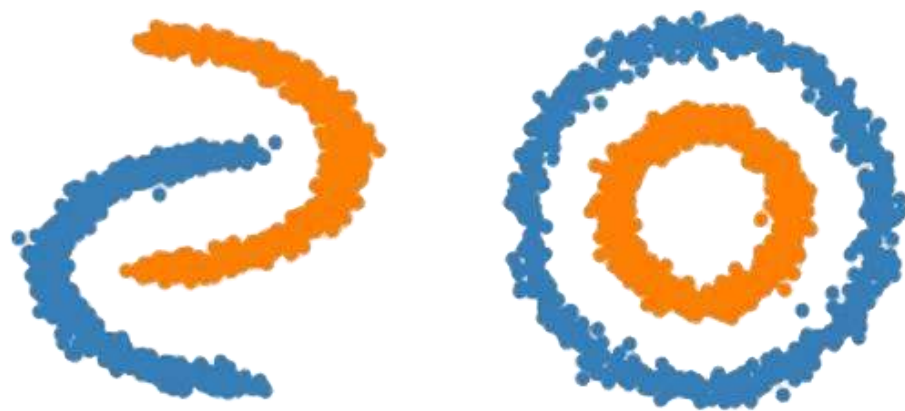
性能度量 - 内部指标

- ❑ **Calinski_harabaz Index (CH指数)**: Calinski-Harabasz分数值越大则聚类效果越好
- ❑ 类别内部数据的协方差越小越好，类别之间的协方差越大越好，这样的Calinski-Harabasz分数会高。在scikit-learn中， Calinski-Harabasz Index对应的方法是 `metrics.calinski_harabaz_score`。在真实的分群label不知道的情况下，可以作为评估模型的一个指标。数值越小可以理解为：组间协方差很小，组与组之间界限不明显。与轮廓系数的对比，最大的优势：快！毫秒级

```
1 >>> import numpy as np
2 >>> from sklearn.cluster import KMeans
3 >>> kmeans_model = KMeans(n_clusters=3, random_state=1).fit(X)
4 >>> labels = kmeans_model.labels_
5 >>> metrics.calinski_harabaz_score(X, labels)
6 560.39...
```

性能度量

- 但对于如下图所示的数据在N维空间中的不是凸分布的情况下，此时我们就需要采用另外的一些评价指标。
- ✓ 典型的无监督聚类算法也很多，例如DBSCAN算法等，在此种情况下的聚类效果就非常的优秀。



大纲

- 聚类任务
- 性能度量
- 距离计算
- 原型聚类
- 密度聚类
- 层次聚类

距离计算

□ 距离度量的性质：

非负性： $dist(x_i, x_j) \geq 0$

同一性： $dist(x_i, x_j) = 0$ 当且仅当 $x_i = x_j$

对称性： $dist(x_i, x_j) = dist(x_j, x_i)$

直递性： $dist(x_i, x_j) \leq dist(x_i, x_k) + dist(x_k, x_j)$

□ 常用距离形式：

给定样本 $\mathbf{x}_i = (x_{i1}; x_{i2}; \dots; x_{in})$ 与 $\mathbf{x}_j = (x_{j1}; x_{j2}; \dots; x_{jn})$

最常用的闵可夫斯基距离 (Minkowski distance)：

$$dist(x_i, x_j) = \left(\sum_{u=1}^n |x_{iu} - x_{ju}|^p \right)^{\frac{1}{p}}$$

$p=2$ 时：欧氏距离 (Euclidean distance) .

$p=1$ 时：曼哈顿距离 (Manhattan distance) .

$p \mapsto \infty$ ：得到切比雪夫距离。

距离度量

- 常将属性划分为“**连续属性**”(continuous attribute)和“**离散属性**”(categorical attribute)。
- 前者在定义域上有无穷多个可能的取值，后者在定义域上是有限个取值。
- 然而，在讨论距离计算时，属性上是否定义了“序”关系更为重要。
- 例如：定义域为{1, 2, 3}的离散属性与连续属性的性质更接近一些，能直接在属性值上计算距离：“1”与“2”比较接近、与“3”比较远，这样的属性称为“**有序属性**”(ordinal attribute);
- 而定义域为{飞机, 火车, 轮船}这样的离散属性则不能直接在属性值上计算距离，称为“**无序属性**”(non-ordinal attribute)。
- 显然，闵可夫斯基距离可用于有序属性。

距离度量

□ 对于**无序(non-ordinal)属性**, 可使用 **Value Difference Metric, VDM**

令 $m_{u,a}$ 表示属性 u 上取值为 a 的样本数, $m_{u,a,i}$ 表示在第 i 个样本簇中在属性 u 上取值为 a 的样本数, k 为样本簇数, 则属性 u 上两个离散值 a 与 b 之间的**VDM**距离为

$$VDM_p(a, b) = \sum_{i=1}^k \left| \frac{m_{u,a,i}}{m_{u,a}} - \frac{m_{u,b,i}}{m_{u,b}} \right|^p \quad \text{i表示类簇}$$

□ 对于**混合属性**, 将闵可夫斯基距离和**VDM**结合进行处理 (可使用**MinkovDM**)

假定有 n_c 个有序属性、 $n-n_c$ 个无序属性, 不失一般性, 令有序属性排列在无序属性之前, 则

$$MinkovDM_p(x_i, x_j) = \left(\underbrace{\sum_{u=1}^{n_c} |x_{iu} - x_{ju}|^p}_{\text{有序属性}} + \underbrace{\sum_{u=n_c+1}^n VDM_p(x_{iu}, x_{ju})}_{\text{无序属性}} \right)^{\frac{1}{p}}$$

聚类方法概述



聚类的“好坏”不存在绝对的标准

The goodness of clustering depends on the opinion of the user.

聚类也许是机器学习中“新算法”出现最多、最快的领域总能找到一个新的“标准”，
使以往算法对它无能为力

常见的聚类算法

□ 原型聚类

- 亦称“基于原型的聚类”(prototype-based clustering)
- 假设：聚类结构能通过一组原型刻画
- 过程：先对原型初始化，然后对原型进行迭代更新求解
- 代表：**k均值聚类**，学习向量量化(LVQ)，高斯混合聚类

□ 密度聚类

- 亦称“基于密度的聚类”(density-based clustering)
- 假设：聚类结构能通过样本分布的紧密程度确定
- 过程：从样本密度的角度来考察样本之间的可连接性，并基于可连接样本不断扩展聚类簇
- 代表：**DBSCAN**, OPTICS, DENCLUE

□ 层次聚类 (hierarchical clustering)

- 假设：能够产生不同粒度的聚类结果
- 过程：在不同层次对数据集进行划分，从而形成树形的聚类结构
- 代表：**AGNES(自底向上)**，DIANA (自顶向下)

大纲

- 聚类任务
- 性能度量
- 距离计算
- 原型聚类
- 密度聚类
- 层次聚类

原型聚类

- 原型聚类：也称为“基于原型的聚类” (prototype-based clustering)，此类算法假设聚类结构能通过一组原型刻画。
- 算法过程：通常情况下，算法先对原型进行初始化，再对原型进行迭代更新求解。
- 接下来，介绍几种著名的原型聚类算法

k 均值算法、学习向量量化算法、高斯混合聚类算法

原型聚类 - k 均值算法

给定数据集 $D = \{x_1, x_2, \dots, x_m\}$, **k 均值 (k-means)** 算法针对聚类所得簇划分 $C = \{C_1, C_2, \dots, C_k\}$ 最小化平方误差

$$E = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|_2^2$$

其中, $\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$ 是簇 C_i 的均值向量。

E 值在一定程度上刻画了簇内样本围绕簇均值向量的紧密程度, E 值越小, 则簇内样本相似度越高。

原型聚类 - k 均值算法

每个簇以该簇中所有样本点的“均值”表示

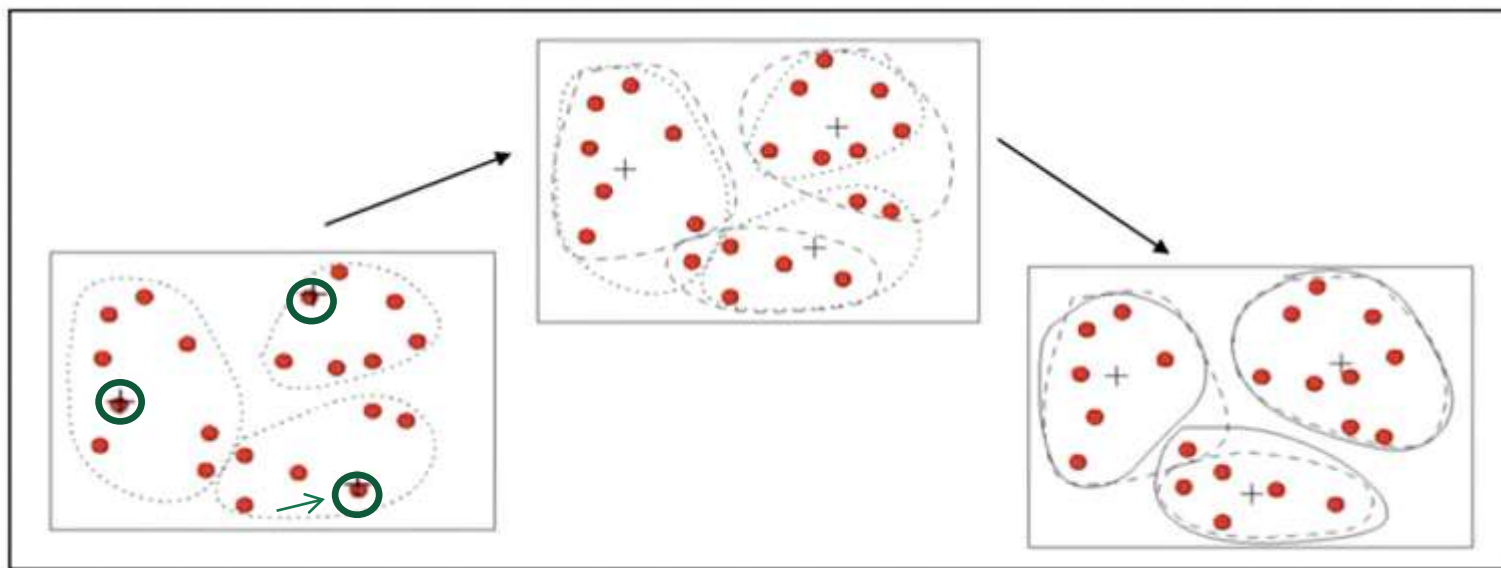
若不以均值向量为原型，而是以距离它最近的样本点为原型，则得到k-medoids算法

Step1: 随机选取 k 个样本点作为簇中心

Step2: 将其他样本点根据其与簇中心的距离，划分给最近的簇

Step3: 更新各簇的均值向量，将其作为新的簇中心

Step4: 若所有簇中心未发生改变，则停止；否则执行 Step 2



原型聚类 - k 均值算法

□ k 均值算法采用了贪心策略，通过迭代优化来近似求解上式：

```
输入: 样本集  $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$ ;  
      聚类簇数  $k$ .  
过程:  
1: 从  $D$  中随机选择  $k$  个样本作为初始均值向量  $\{\mu_1, \mu_2, \dots, \mu_k\}$   
2: repeat  
3:   令  $C_i = \emptyset$  ( $1 \leq i \leq k$ )  
4:   for  $j = 1, \dots, m$  do  
5:     计算样本  $\mathbf{x}_j$  与各均值向量  $\mu_i$  ( $1 \leq i \leq k$ ) 的距离:  $d_{ji} = \|\mathbf{x}_j - \mu_i\|_2$ ;  
6:     根据距离最近的均值向量确定  $\mathbf{x}_j$  的簇标记:  $\lambda_j = \arg \min_{i \in \{1, 2, \dots, k\}} d_{ji}$ ;  
7:     将样本  $\mathbf{x}_j$  划入相应的簇:  $C_{\lambda_j} = C_{\lambda_j} \cup \{\mathbf{x}_j\}$ ;  
8:   end for  
9:   for  $i = 1, \dots, k$  do  
10:    计算新均值向量:  $\mu'_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x}$ ;  
11:    if  $\mu'_i \neq \mu_i$  then  
12:      将当前均值向量  $\mu_i$  更新为  $\mu'_i$   
13:    else  
14:      保持当前均值向量不变  
15:    end if  
16:  end for  
17: until 当前均值向量均未更新  
18: return 簇划分结果  
输出: 簇划分  $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$ 
```

其中：

- ✓ 第1 行对均值向量进行初始化，
- ✓ 在第4 - 8 行与第9 - 16行依次对当前簇划分及均值向量迭代更新，
- ✓ 若迭代更新后聚类结果保持不变，则 在第18行将当前簇划分结果返回。

为避免运行时间过长，通常设置一个最大运行数或最小调整幅度阈值，若达到最大轮数或调整当前均值向量均未更新度小于阈值，则停止运行。

k均值（k-means）算法

□ k均值算法实例

接下来以表9-1的西瓜数据集4.0为例，来演示k均值算法的学习过程。将编号为*i* 的样本称为 x_i 。
是一个包含“密度”与“含糖率”两个属性值的二维向量。

表9-1：西瓜数据集4.0

编号	密度	含糖率	编号	密度	含糖率	编号	密度	含糖率
1	0.697	0.460	11	0.245	0.057	21	0.748	0.232
2	0.774	0.376	12	0.343	0.099	22	0.714	0.346
3	0.634	0.264	13	0.639	0.161	23	0.483	0.312
4	0.608	0.318	14	0.657	0.198	24	0.478	0.437
5	0.556	0.215	15	0.360	0.370	25	0.525	0.369
6	0.403	0.237	16	0.593	0.042	26	0.751	0.489
7	0.481	0.149	17	0.719	0.103	27	0.532	0.472
8	0.437	0.211	18	0.359	0.188	28	0.473	0.376
9	0.666	0.091	19	0.339	0.241	29	0.725	0.445
10	0.243	0.267	20	0.282	0.257	30	0.446	0.459

样本 9~21 的类别是“好瓜=否”，其他样本的类别是“好瓜=是”。由于本节使用无标记样本，因此类别标记信息未在表中给出。

k 均值 (k -means) 算法

□ k 均值算法实例

假定聚类簇数 $k = 3$ ，算法开始时，随机选择3个样本 x_6, x_{12}, x_{27} 作为初始均值向量，即

$$\mu_1 = (0.403; 0.237), \mu_2 = (0.343; 0.099), \mu_3 = (0.533; 0.472)$$

考察样本 $x_1 = (0.697; 0.460)$ ，它与当前均值向量 μ_1, μ_2, μ_3 的距离分别为 **0.369, 0.506, 0.166**，因此 x_1 将被划入簇 C_3 中。类似的，对数据集中的所有样本考察一遍后，可得当前簇划分为

$$C_1 = \{x_5, x_6, x_7, x_8, x_9, x_{10}, x_{13}, x_{14}, x_{15}, x_{17}, x_{18}, x_{19}, x_{20}, x_{23}\}$$

$$C_2 = \{x_{11}, x_{12}, x_{16}\}$$

$$C_3 = \{x_1, x_2, x_3, x_4, x_{21}, x_{22}, x_{24}, x_{25}, x_{26}, x_{27}, x_{28}, x_{29}, x_{30}\}$$

于是，可以从 C_1, C_2, C_3 分别求得新的均值向量

$$\mu'_1 = (0.473; 0.214), \mu'_2 = (0.394; 0.066), \mu'_3 = (0.623; 0.388)$$

更新当前均值向量后，不断重复上述过程，如下页图例所示。

第五轮迭代产生的结果与第四轮迭代相同，于是算法停止，得到最终的簇划分。

k 均值 (k -means) 算法

□ 聚类结果:

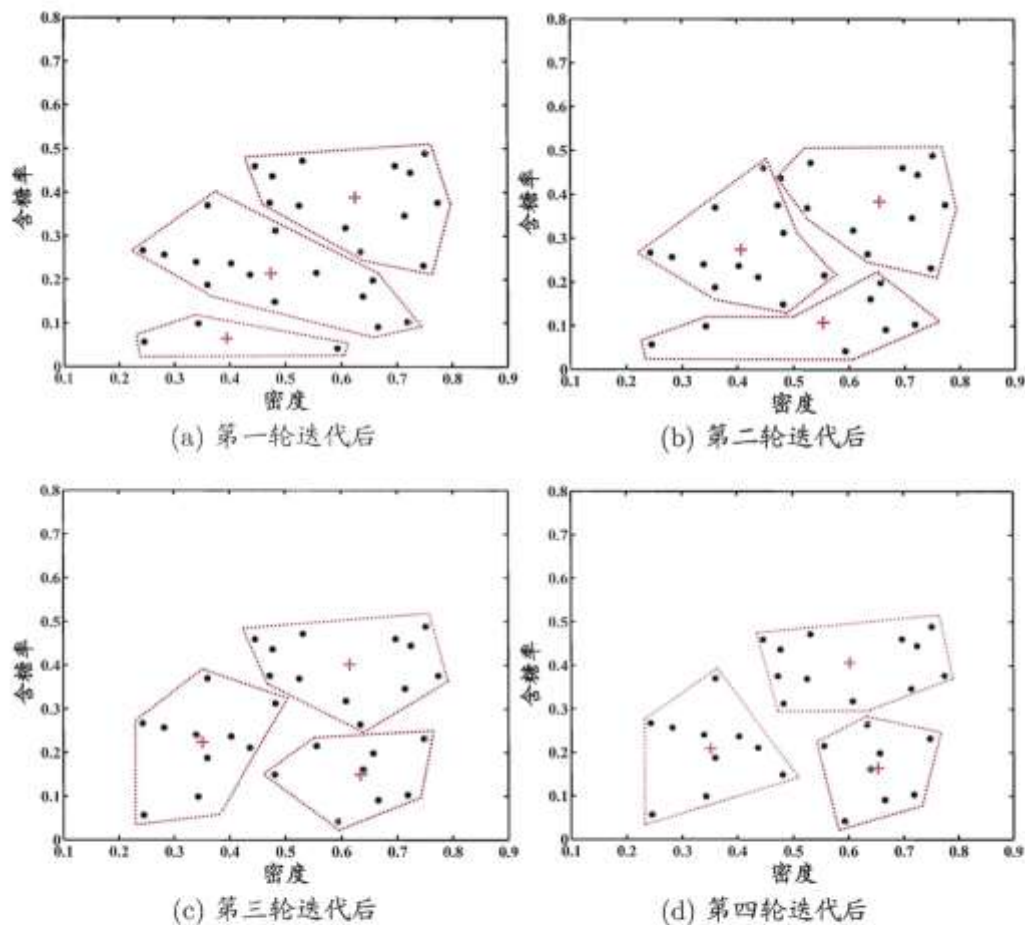


图 9.3 西瓜数据集 4.0 上 k 均值算法 ($k=3$) 在各轮迭代后的结果。样本点与均值向量分别用“•”与“+”表示, 红色虚线显示出簇划分。

k-means算法分析

□ 主要优点:

- 是解决聚类问题的一种经典算法，简单、快速；
- 对处理大数据集，该算法是相对可伸缩和高效率的。其时间复杂度为 $O(nkt)$ ， n 是对象数目， k 是簇的数目， t 是迭代次数。通常 $k \ll n$ & $t \ll n$ ；
- 当结果簇是密集的，而簇与簇之间区别明显时，它的效果较好。

□ 主要缺点:

- 必须事先确定 k 值，很多情况下并不清楚类别个数。但是聚类结果对初始值敏感，对于不同的初始值，可能导致不能结果；
- 初始聚类中心的选择对聚类结果有较大的影响。初值选择不当，可能无法得到好的聚类结果。可以多设一些不同初值，对比最后的运算结果，一直到结果趋于稳定来解决这一问题，但比较耗时耗资源；
- 在簇的平均值被定义的情况下才能使用，这对于处理符号属性的数据不适用；
- 对于噪声和孤立点数据是敏感的，少量的该类数据能够对平均值产生较大的影响

原型聚类-学习向量量化(LVQ)

- 学习向量量化也是试图找到一组原型向量来刻画聚类结构，但假设数据样本带有类别标记，学习过程中利用样本的这些监督信息来辅助聚类，实际上是通过聚类来形成类别的“子类”结构，每个子类对应一个聚类簇。
- 给定样本集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$ ，每个样本 $\mathbf{x}_j$ 是由 n 个属性描述的特征向量 $(x_{j1}; x_{j2}; \dots; x_{jn})$
- $y_j \in \mathcal{Y}$ 是样本 $\mathbf{x}_j$ 的类别标记。LVQ的目标是学得一组 n 维原型向量 $\{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_q\}$ 。每个原型向量代表一个聚类簇，簇标记 $t_i \in \mathcal{Y}$ 。

输入: 样本集 $D = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\}$;
原型向量个数 q , 各原型向量预设的类别标记 $\{t_1, t_2, \dots, t_q\}$;
学习率 $\eta \in (0, 1)$.

过程:

1: 初始化一组原型向量 $\{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_q\}$

2: **repeat**

3: 从样本集 D 随机选取样本 $(\mathbf{x}_j, y_j)$;

4: 计算样本 $\mathbf{x}_j$ 与 $\mathbf{p}_i$ ($1 \leq i \leq q$) 的距离: $d_{ji} = \|\mathbf{x}_j - \mathbf{p}_i\|_2$;

5: 找出与 $\mathbf{x}_j$ 距离最近的原型向量; $i^* = \arg \min_{i \in \{1, 2, \dots, q\}} d_{ji}$;

6: **if** $y_j = t_{i^*}$ **then**

7: $\mathbf{p}' = \mathbf{p}_{i^*} + \eta \cdot (\mathbf{x}_j - \mathbf{p}_{i^*})$

$\mathbf{x}_j$ 与 $\mathbf{p}_{i^*}$ 的类别相同

8: **else**

9: $\mathbf{p}' = \mathbf{p}_{i^*} - \eta \cdot (\mathbf{x}_j - \mathbf{p}_{i^*})$

$\mathbf{x}_j$ 与 $\mathbf{p}_{i^*}$ 的类别不同

10: **end if**

11: 将原型向量 $\mathbf{p}_{i^*}$ 更新为 $\mathbf{p}'$

12: **until** 满足停止条件

13: **return** 当前原型向量

输出: 原型向量 $\{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_q\}$

学习向量量化

□ 聚类效果:

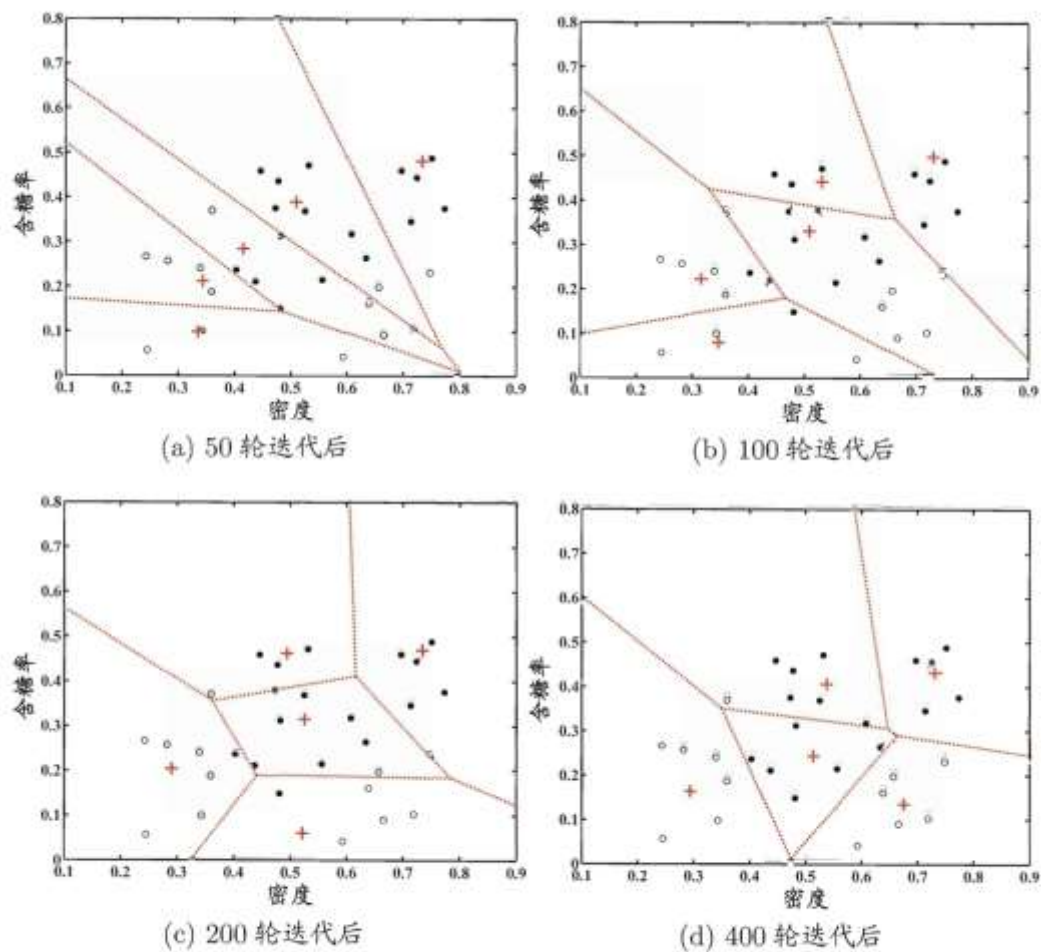


图 9.5 西瓜数据集 4.0 上 LVQ 算法($q=5$)在不同轮数迭代后的聚类结果. c_1 , c_2 类样本点与原型向量分别用“●”, “o”与“+”表示, 红色虚线显示出聚类形成的 Voronoi 划分.

原型聚类-混合高斯聚类(GMM)

高斯混合 (Mixture-of-Gaussian) 聚类：采用概率模型来表达聚类原型。

对 n 维样本空间中的随机向量 x ，若 x 服从高斯分布，其概率密度函数为

$$p(x) = \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma|^{\frac{1}{2}}} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}$$

□ 假设样本由以下高斯混合分布组成

$$p_M(x) = \sum_{i=1}^k \alpha_i p(x | \mu_i, \Sigma_i)$$

生成式模型

- 首先，根据 $\alpha_1, \alpha_2, \dots, \alpha_k$ 定义的先验分布选择高斯混合成分，其中 α_i 为选择第 i 个混合成分的概率；
- 然后，根据被选择的混合成分的概率密度函数进行采样，从而生成相应的样本

混合高斯聚类

- 样本 x_j 由第 i 个高斯混合成分生成的后验概率为

$$p_M(z_j = i | x_j) = \frac{P(z_j = i) \cdot p_M(x_j | z_j = i)}{p_M(x_j)} = \frac{\alpha_i \cdot p(x_j | \mu_i, \Sigma_i)}{\sum_{l=1}^k \alpha_l p(x_j | \mu_l, \Sigma_l)} \quad (9.30)$$

简记为 $\gamma_{ji} (i = 1, 2, \dots, k)$

- 模型求解：最大化（对数）似然（防止下溢）

$$LL(D) = \ln \left(\prod_{j=1}^m p_M(x_j) \right) = \sum_{j=1}^m \ln \left(\sum_{i=1}^k \alpha_i \cdot p(x_j | \mu_i, \Sigma_i) \right)$$

EM 算法：

- (E步) 根据当前参数计算每个样本属于每个高斯成分的后验概率 γ_{ji}
- (M步) 更新模型参数 $\{(\alpha_i, \mu_i, \Sigma_i) \mid 1 \leq i \leq k\}$

高斯混合聚类—模型求解

□ 模型求解：最大化（对数）似然(防止下溢)

$$LL(D) = \ln \left(\prod_{j=1}^m p_M(x_j) \right) = \sum_{j=1}^m \ln \left(\sum_{i=1}^k \alpha_i \cdot p(x_j | \mu_i, \Sigma_i) \right)$$

令：

$$\frac{\partial LL(D)}{\partial \mu_i} = 0 \quad \longrightarrow \quad \mu_i = \frac{\sum_{j=1}^m \gamma_{ji} x_j}{\sum_{j=1}^m \gamma_{ji}}$$

$$\frac{\partial LL(D)}{\partial \Sigma_i} = 0 \quad \longrightarrow \quad \Sigma_i = \frac{\sum_{j=1}^m \gamma_{ji} (x_j - \mu_i)(x_j - \mu_i)^T}{\sum_{j=1}^m \gamma_{ji}}$$

高斯混合聚类

□ 算法伪代码:

输入: 样本集 $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$;
高斯混合成分个数 k .

过程:

1: 初始化高斯混合分布的模型参数 $\{(\alpha_i, \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) \mid 1 \leq i \leq k\}$

2: **repeat**

3: **for** $j = 1, \dots, m$ **do**

4: 根据(9.30)计算 $\mathbf{x}_j$ 由各混合成分生成的后验概率, 即

$$\gamma_{ji} = p_{\mathcal{M}}(z_j = i \mid \mathbf{x}_j) \quad (1 \leq i \leq k)$$

5: **end for**

6: **for** $i = 1, \dots, k$ **do**

7: 计算新均值向量: $\boldsymbol{\mu}'_i = \frac{\sum_{j=1}^m \gamma_{ji} \mathbf{x}_j}{\sum_{j=1}^m \gamma_{ji}};$

8: 计算新协方差矩阵: $\boldsymbol{\Sigma}'_i = \frac{\sum_{j=1}^m \gamma_{ji} (\mathbf{x}_j - \boldsymbol{\mu}'_i)(\mathbf{x}_j - \boldsymbol{\mu}'_i)^\top}{\sum_{j=1}^m \gamma_{ji}};$

9: 计算新混合系数: $\alpha'_i = \frac{\sum_{j=1}^m \gamma_{ji}}{m};$

10: **end for**

11: 将模型参数 $\{(\alpha_i, \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) \mid 1 \leq i \leq k\}$ 更新为 $\{(\alpha'_i, \boldsymbol{\mu}'_i, \boldsymbol{\Sigma}'_i) \mid 1 \leq i \leq k\}$

12: **until** 满足停止条件

13: $C_i = \emptyset \quad (1 \leq i \leq k)$

14: **for** $j = 1, \dots, m$ **do**

15: 根据(9.31)确定 $\mathbf{x}_j$ 的簇标记 λ_j ;

$$\lambda_j = \arg \max_{i \in \{1, 2, \dots, k\}} \gamma_{ji}$$

16: 将 $\mathbf{x}_j$ 划入相应的簇: $C_{\lambda_j} = C_{\lambda_j} \cup \{\mathbf{x}_j\}$

17: **end for**

18: **return** 簇划分结果

输出: 簇划分 $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$

高斯混合聚类

以表 9.1 的西瓜数据集 4.0 为例, 令高斯混合成分的个数 $k = 3$. 算法开始时, 假定将高斯混合分布的模型参数初始化为: $\alpha_1 = \alpha_2 = \alpha_3 = \frac{1}{3}$; $\mu_1 = \mathbf{x}_6$, $\mu_2 = \mathbf{x}_{22}$, $\mu_3 = \mathbf{x}_{27}$; $\Sigma_1 = \Sigma_2 = \Sigma_3 = \begin{pmatrix} 0.1 & 0.0 \\ 0.0 & 0.1 \end{pmatrix}$.

在第一轮迭代中, 先计算样本由各混合成分生成的后验概率. 以 $\mathbf{x}_1$ 为例, 由式(9.30)算出后验概率 $\gamma_{11} = 0.219$, $\gamma_{12} = 0.404$, $\gamma_{13} = 0.377$. 所有样本的后验概率算完后, 得到如下新的模型参数:

$$\alpha'_1 = 0.361, \alpha'_2 = 0.323, \alpha'_3 = 0.316$$

$$\mu'_1 = (0.491; 0.251), \mu'_2 = (0.571; 0.281), \mu'_3 = (0.534; 0.295)$$

$$\Sigma'_1 = \begin{pmatrix} 0.025 & 0.004 \\ 0.004 & 0.016 \end{pmatrix}, \Sigma'_2 = \begin{pmatrix} 0.023 & 0.004 \\ 0.004 & 0.017 \end{pmatrix}, \Sigma'_3 = \begin{pmatrix} 0.024 & 0.005 \\ 0.005 & 0.016 \end{pmatrix}$$

模型参数更新后, 不断重复上述过程, 不同轮数之后的聚类结果如图 9.7 所示.

高斯混合聚类

- $K=3$
- 均值向量 +

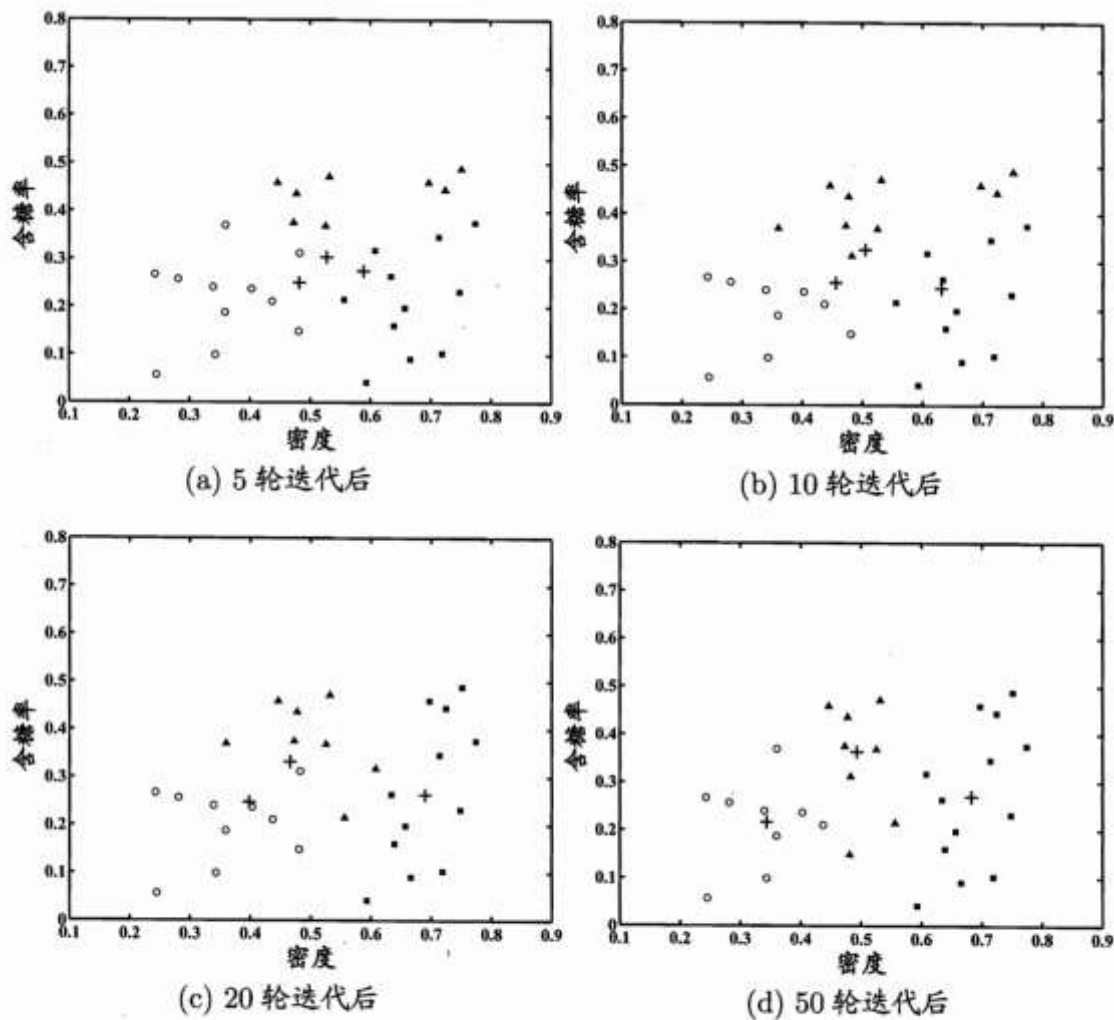


图 9.7 高斯混合聚类($k=3$)在不同轮数迭代后的聚类结果. 其中样本簇 C_1 , C_2 与 C_3 中的样本点分别用“o”, “■”与“▲”表示, 各高斯混合成分的均值向量用“+”表示.

高斯混合聚类

```
import numpy as np
from sklearn import datasets
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture

#读取数据
iris=datasets.load_iris()
x=iris.data[:, :2]
y=iris.target
mu = np.array([np.mean(x[y == i], axis=0) for i in range(3)])
print '实际均值 = \n', mu

#K-Means
kmeans=KMeans(n_clusters=3, init='k-means++', random_state=0)
y_hat1=kmeans.fit_predict(x)
mu1=np.array([np.mean(x[y_hat1 == i], axis=0) for i in range(3)])
print 'K-Means均值 = \n', mu1
print '分类正确率为', np.mean(y_hat1==y)

gmm=GaussianMixture(n_components=3, covariance_type='full', random_state=0)
gmm.fit(x)
print 'GMM均值 = \n', gmm.means_
y_hat2=gmm.predict(x)
print '分类正确率为', np.mean(y_hat2==y)
```

大纲

- 聚类任务
- 性能度量
- 距离计算
- 原型聚类
- 密度聚类
- 层次聚类

基于密度的聚类 (DBSCAN)

□ 密度聚类的定义

密度聚类也称为“基于密度的聚类” (density-based clustering)。此类算法假设聚类结构能通过样本分布的紧密程度来确定。

通常情况下，密度聚类算法从样本密度的角度来考察样本之间的可连接性，并基于可连接样本不断扩展聚类簇来获得最终的聚类结果。

接下来介绍**DBSCAN**这一密度聚类算法。

基于密度的聚类 (DBSCAN)

- DBSCAN (Density-Based Spatial Clustering of Application with Noise, 具有噪声的基于密度的空间聚类应用) 是一种基于高密度连接区域的密度聚类算法。该算法将具有足够高密度的区域划分为簇, 并可以在带有“噪声”的空间数据库中发现任意形状的聚类, 它定义簇为密度相连的点的最大集合。
- 对DBSCAN的基本概念:
 - 对于簇中的任意一个点, 它周围局部点密度必须超过某阈值;
 - 簇中的点在空间上是相互关联的。

基于密度的聚类 (DBSCAN)

□ DBSCAN算法：基于一组“邻域”参数 $(\epsilon, MinPts)$ 来刻画样本分布的紧密程度。给定数据集 $D = \{x_1, x_2, \dots, x_m\}$ 。

□ 定义如下基本概念：

- **ϵ 邻域**：对样本 $x_j \in D$ 其 ϵ 邻域包含样本集 D 中与 x_j 的距离不大于 ϵ 的样本；
即 $N_\epsilon(x_j) = \{x_i \in D \mid \text{dist}(x_i, x_j) \leq \epsilon\}$
- **核心对象**：若样本 x_j 的 ϵ 邻域至少包含 $MinPts$ 个样本，即 $|N_\epsilon(x_j)| \geq MinPts$ ，则该样本点为一个核心对象；
- **密度直达**：若样本 x_j 位于样本 x_i 的 ϵ 邻域中，且 x_i 是一个核心对象，则称样本 x_j 由 x_i 密度直达；
- **密度可达**：对样本 x_i 与 x_j ，若存在样本序列 $p_1, p_2, \dots, p_n$ ，其中 $p_1 = x_i, p_n = x_j$
且 p_{i+1} 由 p_i 密度直达，则 x_j 由 x_i 密度可达；
- **密度相连**：对样本 x_i 与 x_j ，若存在样本 x_k 使得两样本均由 x_k 密度可达，则称该两样本密度相连。

DBSCAN算法

- 任意点 p 的局部点密度由两个参数定义，即 Eps （邻域的最大半径）及 $MinPts$ （在 Eps 邻域中的最少点数）。相关一些概念的定义如下：

定义1 (Eps 邻域)

给定一个对象 p ， p 的 Eps 邻域 $N_{Eps}(p)$ 定义为以 p 为核心，以 Eps 为半径的 d 维超球体区域，即：

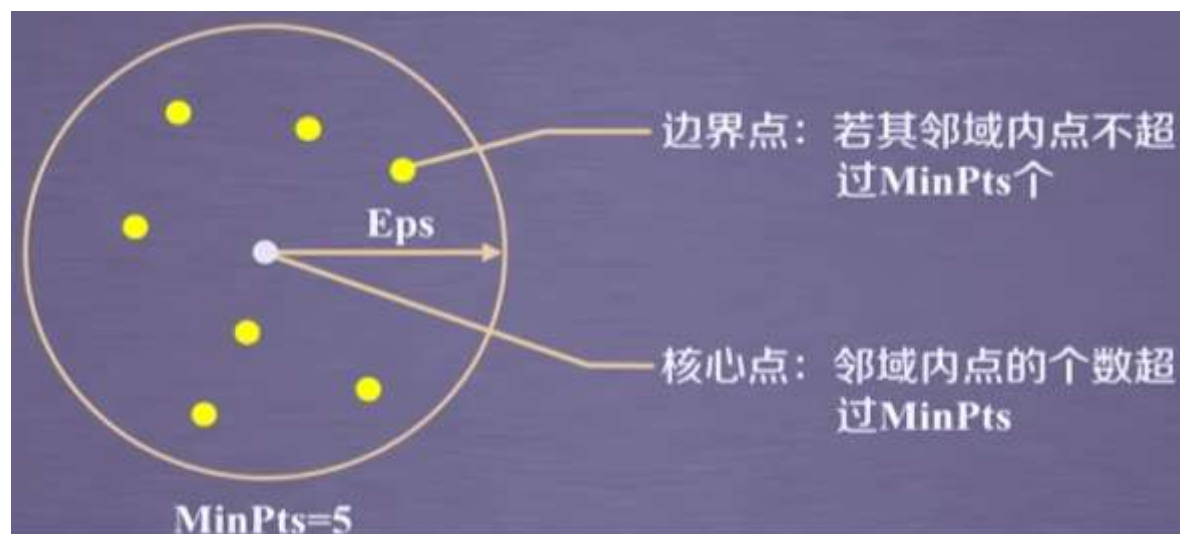
$$N_{Eps}(p) = \{q \in D \mid dist(p, q) \leq Eps\}$$

其中， D 为 d 维实空间上的数据集， $dist(p, q)$ 表示 D 中的2个对象 p 和 q 之间的距离。

DBSCAN算法

定义2（核心点与边界点）

对于对象 $p \in D$ ，给定一个整数 $MinPts$ ，如果 p 的 Eps 邻域内的对象数满足 $|N_{Eps}(p)| \geq MinPts$ ，则称 p 为 $(Eps, MinPts)$ 条件下的**核心点**；不是核心点但落在某个核心点的 Eps 邻域内的对象称为**边界点**。



DBSCAN算法

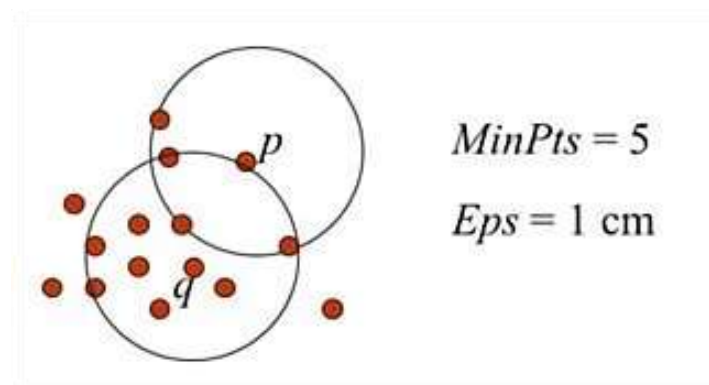
定义3（直接密度可达）

如图所示，给定 $(Eps, MinPts)$ ，
如果对象 p 和 q 同时满足如下条件：

$$p \in N_{Eps}(q) ;$$

$$|N_{Eps}(q)| \geq MinPts \quad (\text{即 } q \text{ 是核心点}) ,$$

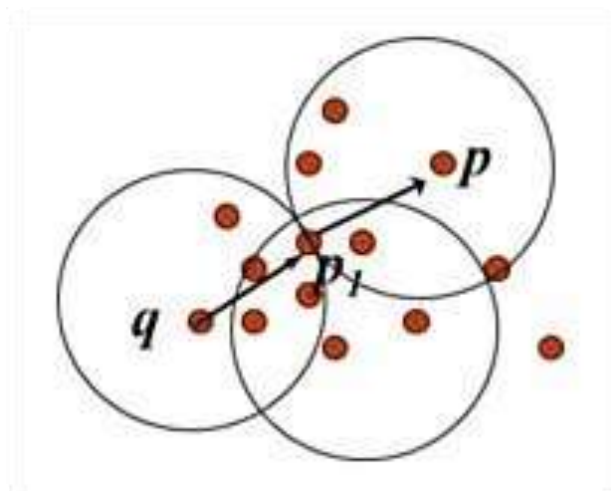
则称对象 p 是从对象 q 出发，直接密度可达的。



DBSCAN算法

定义4（密度可达）

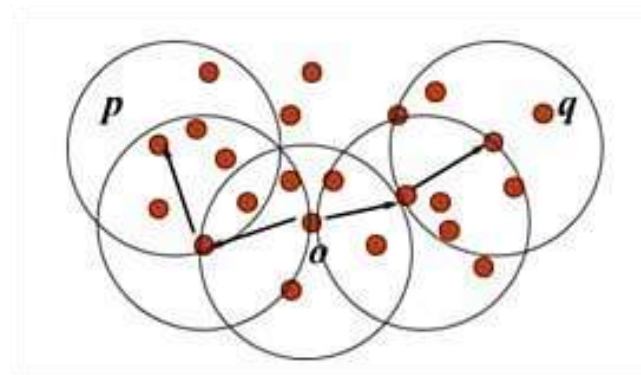
如图所示，给定数据集 D ，当存在一个对象链 $p_1, p_2, p_3, \dots, p_n$ ，其中 $p_1 = q$ ， $p_n = p$ ，对于 $p_i \in D$ ，如果在条件(Eps , $MinPts$)下 p_{i+1} 从 p_i 直接密度可达，则称对象 p 从对象 q 在条件 (Eps , $MinPts$)下密度可达。密度可达是非对称的，即 p 从 q 密度可达不能推出 q 也从 p 密度可达。



DBSCAN算法

定义5 (密度相连)

如图所示，如果数据集 D 中存在一个对象 o ，使得对象 p 和 q 是从 o 在 $(Eps, MinPts)$ 条件下密度可达的，那么称对象 p 和 q 在 $(Eps, MinPts)$ 条件下密度相连。密度相连是对称的。



密度聚类

□ 一个例子

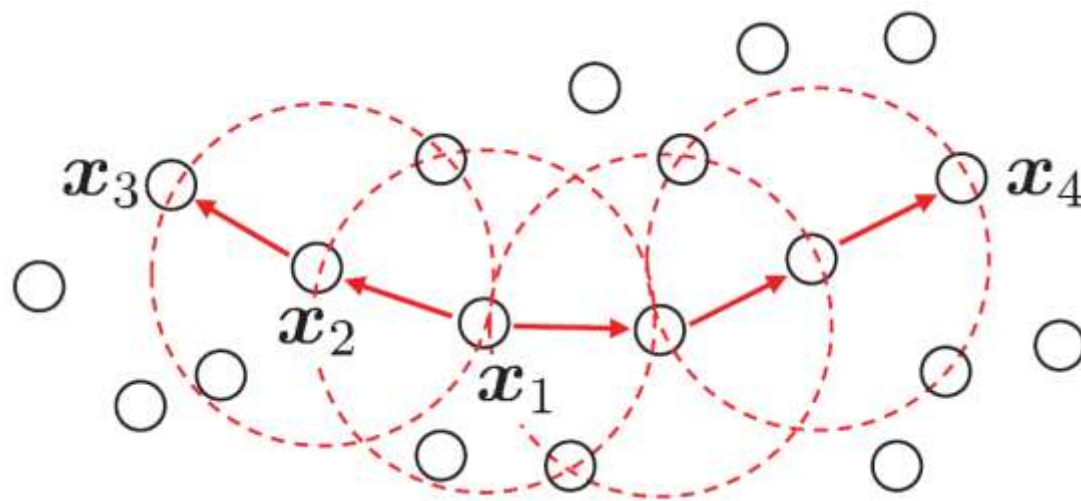
令 $MinPts = 3$ ，则
虚线显示出 ϵ 领域。

x_1 是核心对象。

x_2 由 x_1 密度直达。

x_3 由 x_1 密度可达。

x_3 与 x_4 密度相连。



DBSCAN算法

□ 对“簇”的定义

由密度可达关系导出的最大密度相连样本集合。

□ 对“簇”的形式化描述

给定领域参数 $(\epsilon, MinPts)$ ，簇是满足以下性质的非空样本子集：

连接性： $x_i \in C, x_j \in C \Rightarrow x_i$ 与 x_j 密度相连

最大性： $x_i \in C, x_i$ 与 x_j 密度可达 $\Rightarrow x_j \in C$

实际上，若 x 为核心对象，由 x 密度可达的所有样本组成的集合记为 $X = \{x' \in D \mid x' \text{ 由 } x \text{ 密度可达}\}$ ，则 X 为满足连接性与最大性的簇。

DBSCAN算法流程

算法： DBSCAN。基于高密度连接区域的密度聚类算法。

输入： Eps 、 $MinPts$ 和包含 n 个对象的数据库。

输出： 基于密度的聚类结果。

方法：

- ①任意选取一个没有加簇标签的点 p ；
- ②得到所有从 p 关于 Eps 和 $MinPts$ 密度可达的点；
- ③如果 p 是一个核心点，形成一个新的簇，给簇内所有对象点加簇标签；
- ④如果 p 是一个边界点，没有从 p 密度可达的点，DBSCAN将访问数据库中的下一个点；
- ⑤继续这一过程，直到数据库中的所有点都被处理。

DBSCAN算法

□ DBSCAN算法伪代码：

于是，DBSCAN算法先任选数据集中的一个核心对象为“种子” (seed), 再由此出发确定相应的聚类簇。

```
输入: 样本集  $D = \{x_1, x_2, \dots, x_m\}$ ;  
      邻域参数  $(\epsilon, MinPts)$ .  
过程:  
1: 初始化核心对象集合:  $\Omega = \emptyset$   
2: for  $j = 1, \dots, m$  do  
3:   确定样本  $x_j$  的  $\epsilon$ -邻域  $N_\epsilon(x_j)$ ;  
4:   if  $|N_\epsilon(x_j)| \geq MinPts$  then  
5:     将样本  $x_j$  加入核心对象集合:  $\Omega = \Omega \cup \{x_j\}$   
6:   end if  
7: end for  
8: 初始化聚类簇数:  $k = 0$   
9: 初始化未访问样本集合:  $\Gamma = D$   
10: while  $\Omega \neq \emptyset$  do  
11:   记录当前未访问样本集合:  $\Gamma_{old} = \Gamma$ ;  
12:   随机选取一个核心对象  $o \in \Omega$ , 初始化队列  $Q = \langle o \rangle$ ;  
13:    $\Gamma = \Gamma \setminus \{o\}$ ;  
14:   while  $Q \neq \emptyset$  do  
15:     取出队列  $Q$  中的首个样本  $q$ ;  
16:     if  $|N_\epsilon(q)| \geq MinPts$  then  
17:       令  $\Delta = N_\epsilon(q) \cap \Gamma$ ;  
18:       将  $\Delta$  中的样本加入队列  $Q$ ;  
19:        $\Gamma = \Gamma \setminus \Delta$ ;  
20:     end if  
21:   end while  
22:    $k = k + 1$ , 生成聚类簇  $C_k = \Gamma_{old} \setminus \Gamma$ ;  
23:    $\Omega = \Omega \setminus C_k$   
24: end while  
25: return 簇划分结果  
输出: 簇划分  $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$ 
```

- ✓ 在第1~7行中，算法先根据给定的邻域参数 $(\epsilon, MinPts)$
- ✓ 找出所有核心对象；
- ✓ 然后在第10~24行中，以任一核心对象为出发点，找出由其密度可达的样本生成聚类簇，直到所有核心对象均被访问过为止。

DBSCAN算法

以表 9.1 的西瓜数据集 4.0 为例, 假定邻域参数 $(\epsilon, MinPts)$ 设置为 $\epsilon = 0.11$, $MinPts = 5$. DBSCAN 算法先找出各样本的 ϵ -邻域并确定核心对象集合: $\Omega = \{x_3, x_5, x_6, x_8, x_9, x_{13}, x_{14}, x_{18}, x_{19}, x_{24}, x_{25}, x_{28}, x_{29}\}$. 然后, 从 Ω 中随机选取一个核心对象作为种子, 找出由它密度可达的所有样本, 这就构成了第一个聚类簇. 不失一般性, 假定核心对象 x_8 被选中作为种子, 则 DBSCAN 生成的第一个聚类簇为

$$C_1 = \{x_6, x_7, x_8, x_{10}, x_{12}, x_{18}, x_{19}, x_{20}, x_{23}\} .$$

然后, DBSCAN 将 C_1 中包含的核心对象从 Ω 中去除: $\Omega = \Omega \setminus C_1 = \{x_3, x_5, x_9, x_{13}, x_{14}, x_{24}, x_{25}, x_{28}, x_{29}\}$. 再从更新后的集合 Ω 中随机选取一个核心对象作为种子来生成下一个聚类簇. 上述过程不断重复, 直至 Ω 为空. 图 9.10 显示出 DBSCAN 先后生成聚类簇的情况. C_1 之后生成的聚类簇为

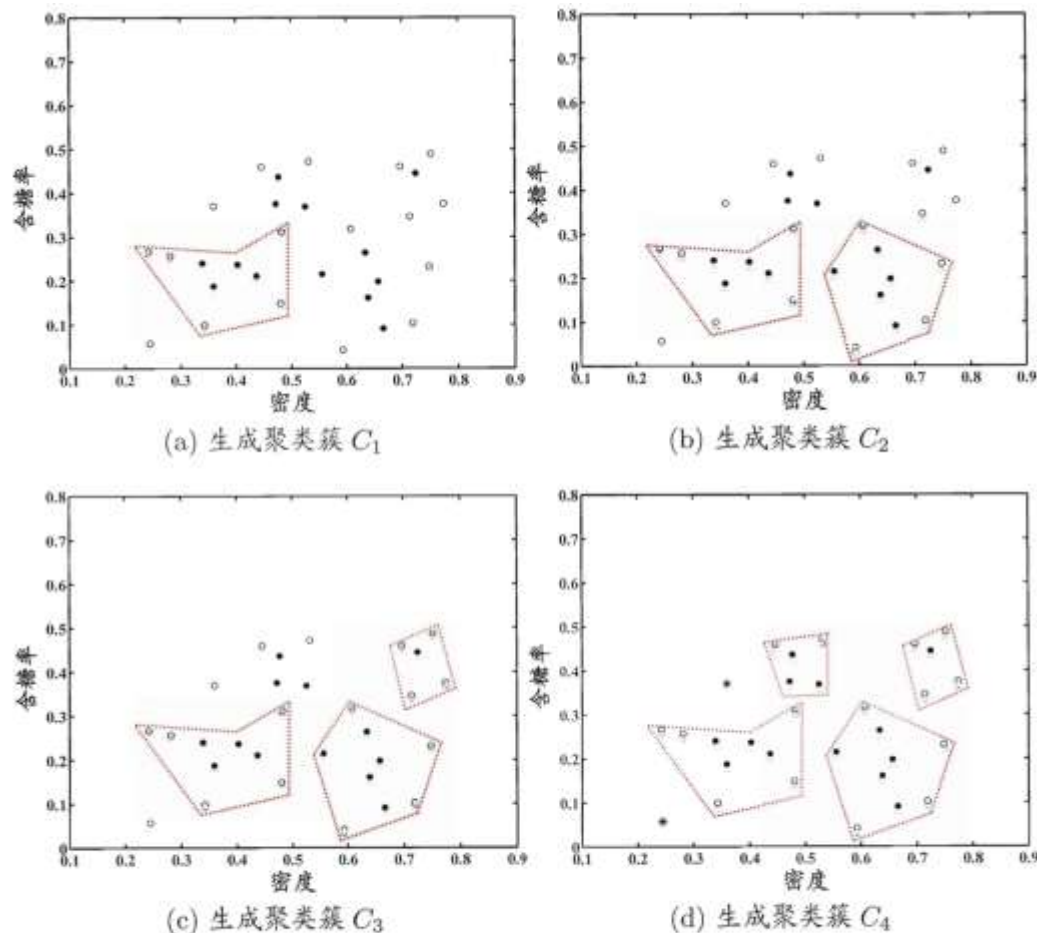
$$C_2 = \{x_3, x_4, x_5, x_9, x_{13}, x_{14}, x_{16}, x_{17}, x_{21}\} ;$$

$$C_3 = \{x_1, x_2, x_{22}, x_{26}, x_{29}\} ;$$

$$C_4 = \{x_{24}, x_{25}, x_{27}, x_{28}, x_{30}\} .$$

DBSCAN算法

核心对象、非核心对象、噪声分别用实心圆、空心圆、星形表示，虚线表示簇的划分



$$C_1 = \{x_6, x_7, x_8, x_{10}, x_{12}, x_{18}, x_{19}, x_{20}, x_{23}\}$$

$$C_2 = \{x_3, x_4, x_5, x_9, x_{13}, x_{14}, x_{16}, x_{17}, x_{21}\};$$

$$C_3 = \{x_1, x_2, x_{22}, x_{26}, x_{29}\};$$

$$C_4 = \{x_{24}, x_{25}, x_{27}, x_{28}, x_{30}\}.$$

图 9.10 DBSCAN 算法($\epsilon = 0.11$, $MinPts = 5$)生成聚类簇的先后情况. 核心对象、非核心对象、噪声样本分别用“●” “○” “*”表示, 红色虚线显示出簇划分.

DBSCAN算法

```
from sklearn.cluster import DBSCAN
```

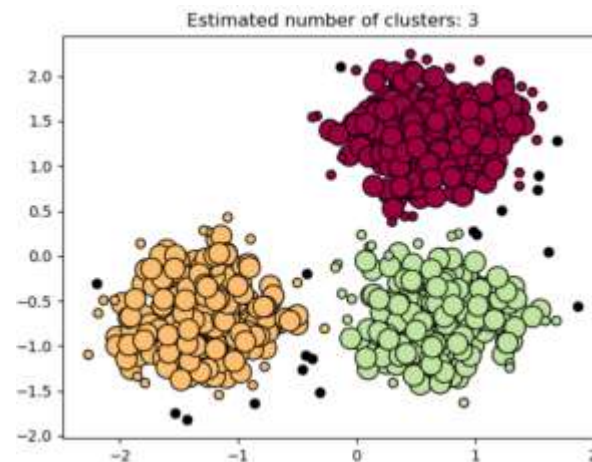
DBSCAN主要参数：

- **eps**: 两个样本被看做邻居节点的最大距离
- **min_samples**: 簇的样本数
- **metric**: 距离计算方式

例如：

```
sklearn.cluster.DBSCAN(eps=0.5, min_samples=5, metric= 'euclidean' )
```

```
from sklearn.cluster import DBSCAN  
  
db = DBSCAN(eps=0.3, min_samples=10).fit(X)
```



DBSCAN算法特点

□ 优点：

- 形成的簇可以具有任意形状和大小；
- 可以自动确定形成的簇数目；
- 可以分离簇和环境噪声；
- 可以被空间索引结构所支持；
- 效率高，即使对大数据集也是如此；
- 一次扫描数据即可完成聚类

□ 缺点：

- 只能发现密度相仿的簇；
- 对用户定义参数（*Eps*和*MinPts*）是敏感的，参数难以确定（特别是对于高维数据），设置的细微不同可能导致差别很大的聚类；
- 所使用参数*Eps*和*MinPts*是两个全局参数，不能刻画高维数据内在的聚类结构，因为真实的高维数据常常具有非常倾斜的分布；
- 计算复杂度至少为 $O(n^2)$ ，若采用空间索引，计算复杂度为 $O(n \log n)$

大纲

- 聚类任务
- 性能度量
- 距离计算
- 原型聚类
- 密度聚类
- 层次聚类

层次聚类

□ 层次聚类试图在不同层次对数据集进行划分，从而形成树形的聚类结构。数据集划分既可采用“自底向上”的聚合策略，也可采用“自顶向下”的分拆策略。

□ **AGNES**算法（自底向上的层次聚类算法）

首先，将样本中的每一个样本看做一个初始聚类簇，然后在算法运行的每一步中找出距离最近的两个聚类簇进行合并，该过程不断重复，直到达到预设的聚类簇的个数。这里的关键是如何计算聚类簇之间的距离。

这里两个聚类簇 C_i 和 C_j 的距离，可以有3种度量方式。

层次聚类

这里两个聚类簇的距离，可以有**3**种度量方式。

容易受极
端值影响

最小距离: $d_{min}(C_i, C_j) = \min_{x \in C_i, z \in C_j} dist(x, z)$

由两个簇的最近样本决定

最大距离: $d_{max}(C_i, C_j) = \max_{x \in C_i, z \in C_j} dist(x, z)$

由两个簇的最远样本决定

平均距离: $d_{avg}(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in C_i} \sum_{z \in C_j} dist(x, z)$

由两个簇的所有样本共同决定

计算量比较大，但结果比前两种方法更合理

AGNES(A Gglomerative NESting)

输入: 样本集 $D = \{x_1, x_2, \dots, x_m\}$;
聚类簇距离度量函数 d ;
聚类簇数 k .

过程:

```
1: for  $j = 1, 2, \dots, m$  do
2:    $C_j = \{x_j\}$ 
3: end for
4: for  $i = 1, 2, \dots, m$  do
5:   for  $j = 1, 2, \dots, m$  do
6:      $M(i, j) = d(C_i, C_j)$ ;
7:      $M(j, i) = M(i, j)$ 
8:   end for
9: end for
10: 设置当前聚类簇个数:  $q = m$ 
11: while  $q > k$  do
12:   找出距离最近的两个聚类簇  $C_{i^*}$  和  $C_{j^*}$ ;
13:   合并  $C_{i^*}$  和  $C_{j^*}$ :  $C_{i^*} = C_{i^*} \cup C_{j^*}$ ;
14:   for  $j = j^* + 1, j^* + 2, \dots, q$  do
15:     将聚类簇  $C_j$  重编号为  $C_{j-1}$ 
16:   end for
17:   删除距离矩阵  $M$  的第  $j^*$  行与第  $j^*$  列;
18:   for  $j = 1, 2, \dots, q - 1$  do
19:      $M(i^*, j) = d(C_{i^*}, C_j)$ ;
20:      $M(j, i^*) = M(i^*, j)$ 
21:   end for
22:    $q = q - 1$ 
23: end while
```

输出: 簇划分 $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$

在第1-9行, 算法先对仅含一个样本的初始聚类簇和相应的距离矩阵进行初始化; 然后在第11-23行, AGNES不断合并距离最近的聚类簇, 并对合并得到的聚类簇的距离矩阵进行更新; 上述过程不断重复, 直至达到预设的聚类簇数。

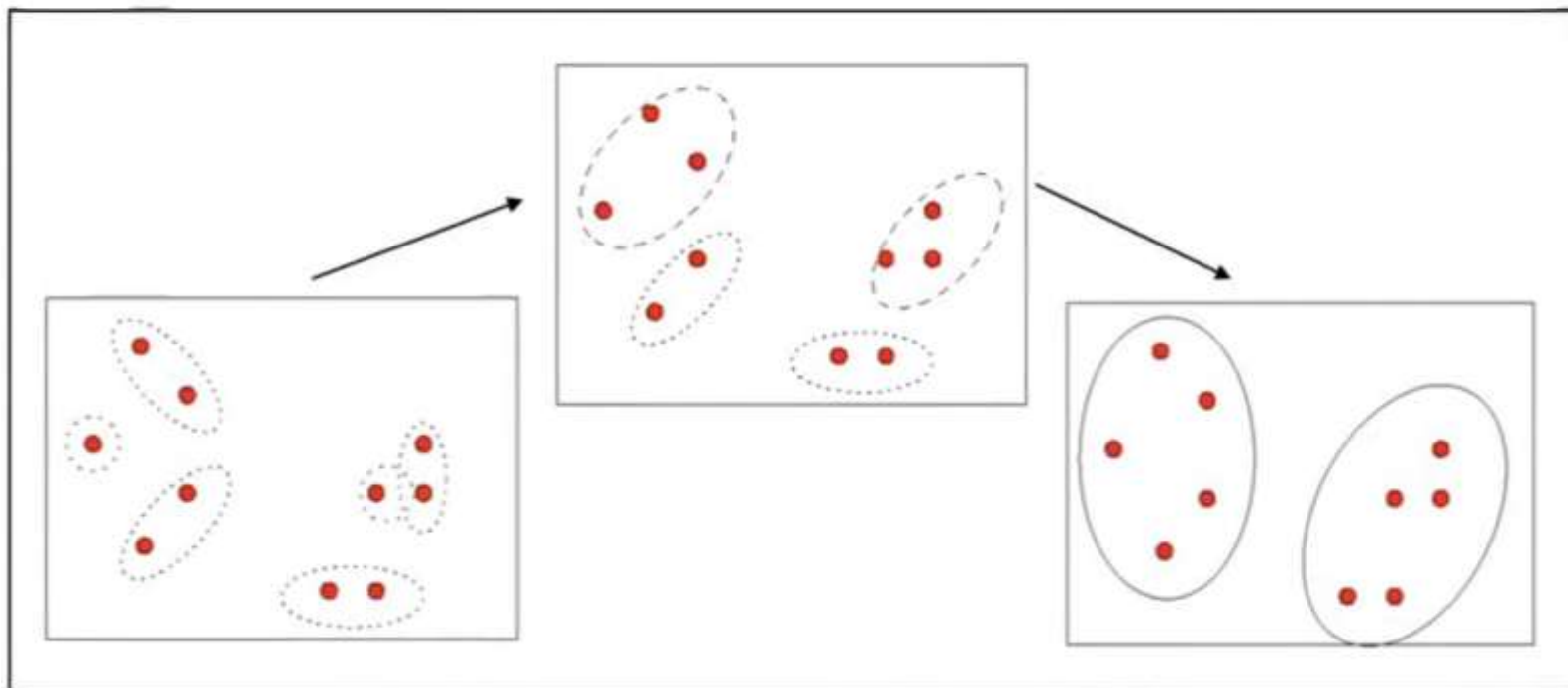
图 9.11 AGNES 算法

AGNES(AGglomerative NESting)

Step1: 将每个样本点作为一个簇

Step2: 合并最近的两个簇

Step3: 若所有样本点都存在与一个簇中，则停止；否则转到 Step2



AGNES(AGglomerative NESting)

□ 西瓜数据集4.0-AGNES算法树状图：

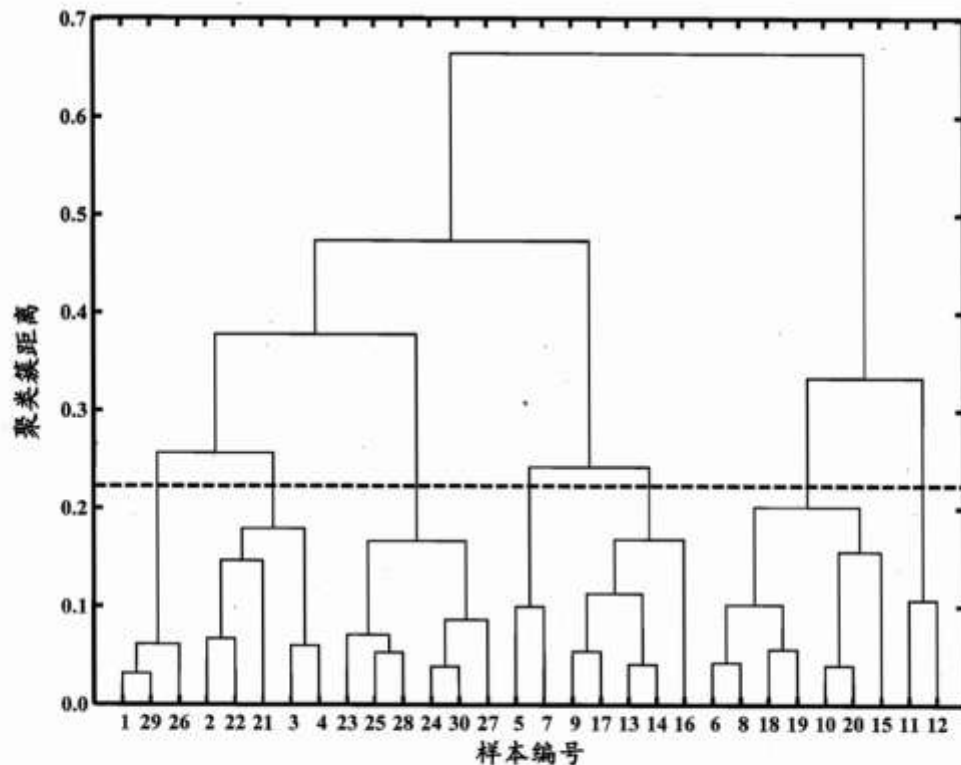


图 9.12 西瓜数据集 4.0 上 AGNES 算法生成的树状图(采用 $d_{\max}$). 横轴对应于样本编号, 纵轴对应于聚类簇距离.

AGNES(AGglomerative NESting)

西瓜数据集4.0-AGNES算法簇的划分：

在树状图的特定层次上进行分割, 则可得到相应的簇划分结果. 例如, 以图 9.12 中所示虚线分割树状图, 将得到包含 7 个聚类簇的结果:

$$C_1 = \{x_1, x_{26}, x_{29}\}; C_2 = \{x_2, x_3, x_4, x_{21}, x_{22}\};$$

$$C_3 = \{x_{23}, x_{24}, x_{25}, x_{27}, x_{28}, x_{30}\}; C_4 = \{x_5, x_7\};$$

$$C_5 = \{x_9, x_{13}, x_{14}, x_{16}, x_{17}\}; C_6 = \{x_6, x_8, x_{10}, x_{15}, x_{18}, x_{19}, x_{20}\};$$

$$C_7 = \{x_{11}, x_{12}\}.$$

层次聚类

□ 将分割层逐步提升，则可得到聚类簇逐渐减少的聚类结果。 $K=7,6,5,4$ 时的聚类效果：

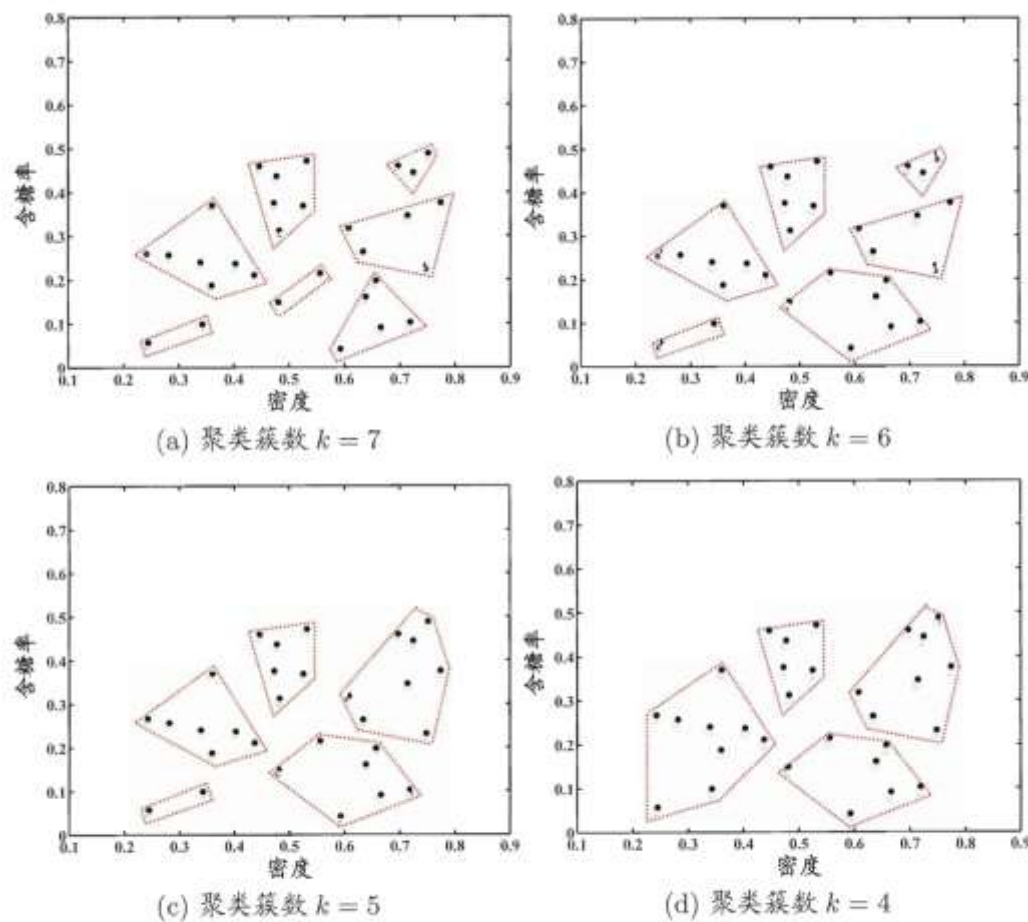


图 9.13 西瓜数据集 4.0 上 AGNES 算法(采用 d_{max})在不同聚类簇数($k = 7, 6, 5, 4$)时的簇划分结果. 样本点用“•”表示, 红色虚线显示出簇划分.

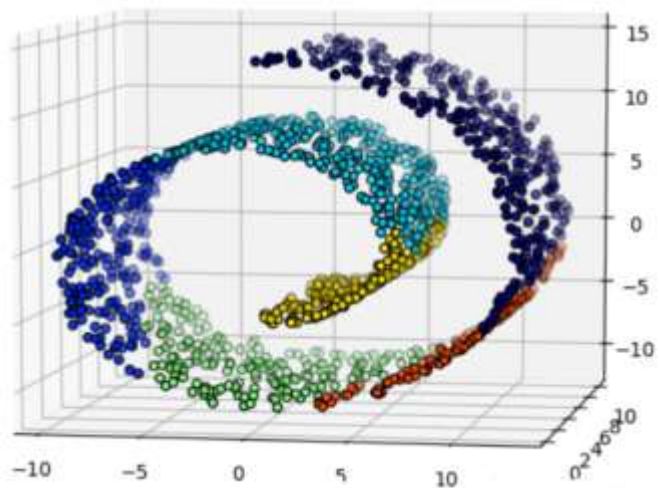
层次聚类

```
import numpy as np
import matplotlib.pyplot as plt
import mpl_toolkits.mplot3d.axes3d as p3
from sklearn.cluster import AgglomerativeClustering
from sklearn.datasets.samples_generator import make_swiss_roll

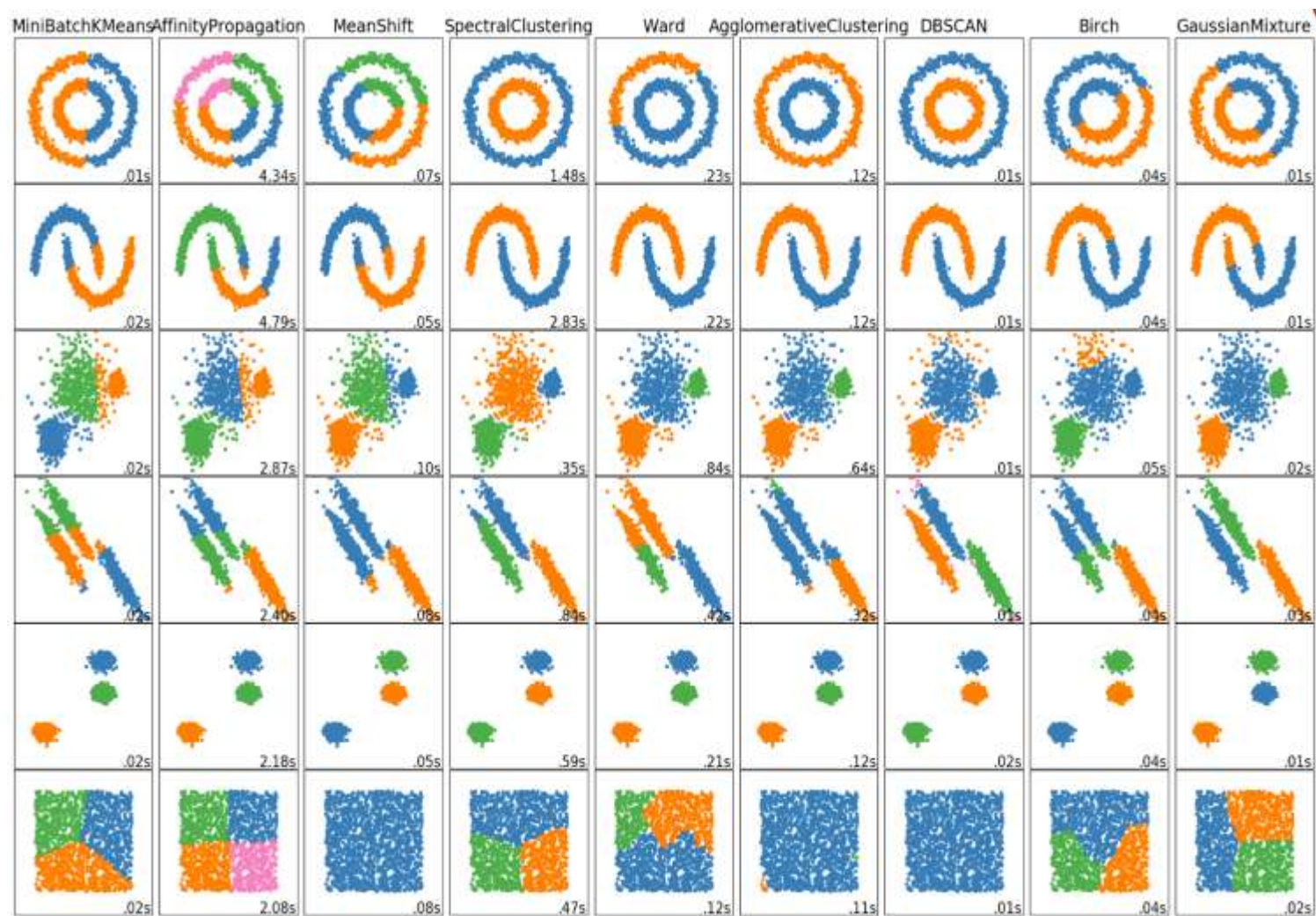
n_samples = 1500
noise = 0.05
X, _ = make_swiss_roll(n_samples, noise) # 卷型数据集
# 进行放缩
X[:, 1] *= .5

ward = AgglomerativeClustering(n_clusters=6, linkage='ward').fit(X)
label = ward.labels_ # 得到label值

fig = plt.figure()
ax = p3.Axes3D(fig)
ax.view_init(7, -80)
for l in np.unique(label):
    ax.scatter(X[label == l, 0], X[label == l, 1], X[label == l, 2],
              color=plt.cm.jet(np.float(l) / np.max(label + 1)),
              s=20, edgecolor='k')
plt.show()
```



sklearn聚类算法比较





重庆大学
CHONGQING UNIVERSITY

THANKS
祝学习快乐!